

Write-up on current state

Currently we have coded and validated Pacejka's magic formula and a Linear brush model with stribeck friction to Luigi's implementation in Matlab and get matching results to machine precision.

In short:

- We have re-run the least-squares fitting with different initial guesses and get results that fit the magic formula model well which can be seen in Figure 2.
- We have tried Luigi's approach of scaling μ_s with μ_d by setting $\mu_s = \mu_d \cdot k_s$ where k_s is fitted during the least squares fit instead of μ_s directly. The results can be seen in Figure 3.
- The least squares solver generally converged to a good optimum close to that of the genetic algorithm and there does not seem to be much room for improvement from the stribeck frictional model when comparing to the magic formula by Pacejka.

Pacejka's magic formula

The magic formula model (lateral) used has coefficients taken from [1] equation 2.114:

$$F_y(\sigma_y) = -1100 \cdot \sin\{1.48 \cdot \arctan[12.27 \cdot \sigma_y - 0.07(12.27 \cdot \sigma_y - \arctan(12.27 \cdot \sigma_y))]\} \quad 1.$$

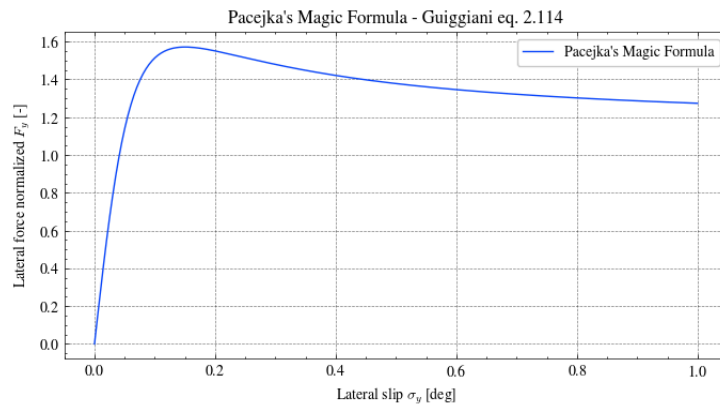


Figure 1: Lateral force predicted by Pacejka's Magic Formula (1) for an FSAE tyre under pure lateral slip.

This is the equation for the lateral force F_y with lateral slip ratio σ_y . The coefficients B, C, D and E are written out explicitly: B = 12.27 is the stiffness factor (slope at zero slip), C = 1.48 is the shape factor, D = 1100 N is the peak force amplitude, and E = 0.07 the curvature factor. These coefficients are experimentally determined and represent the case of pure lateral slip for one specific FSAE tyre.

Linear brush model using least squares fit

The brush model's parameters are fitted using SciPy's non-linear least squares implementation ([link](#)) with finite difference gradients.

For the brush model we are using the brush model solution in the form of:

$$z(x) = -\frac{\text{sgn}_\varepsilon(v)\mu(v)}{\bar{k}_0} \left(1 - \exp\left(-\frac{|v|_\varepsilon \bar{k}_0}{\mu(v)V_r} x\right) \right), \quad \text{where } x \in (0, L) \quad 2.$$

With the frictional coefficient modeled using Stribeck friction.

$$\mu(v) = \mu_d + (\mu_s - \mu_d) \exp\left(-\left(\frac{|v|}{v_S}\right)^{\delta_S}\right) \quad 3.$$

By then fitting the six parameters in Table 1 with the least-squares optimizer different results are obtained based on the bounds and local minima that the optimizer can land in.

Table 1: Brush model parameters for the first fit and the search bounds used during optimisation.

Parameter	Symbol	Units	Physical meaning	Initial guess	Lower bound	Upper bound
Contact patch length	L	m	Length of the tyre-road contact zone	0.1	0.05	0.12
Bristle stiffness	k_0	1/m	Lateral micro-stiffness of the bristles	240	100	800
Dynamic friction	μ_d	-	Friction at high slip speeds	0.7	0.7	2.0
Static friction	μ_s	-	Friction at low slip speeds	1.2	1.0	3.5
Stribeck velocity	v_S	m/s	Characteristic speed for friction transition	3.5	2	20.0
Stribeck exponent	δ_S	-	Sharpness of friction transition	0.6	0.1	2.0

The residuals passed to the optimizer are normalised by $\max(|F_{MF}|)$, so that all residual components are dimensionless and of order one, to ensure equal weighting across the slip range.

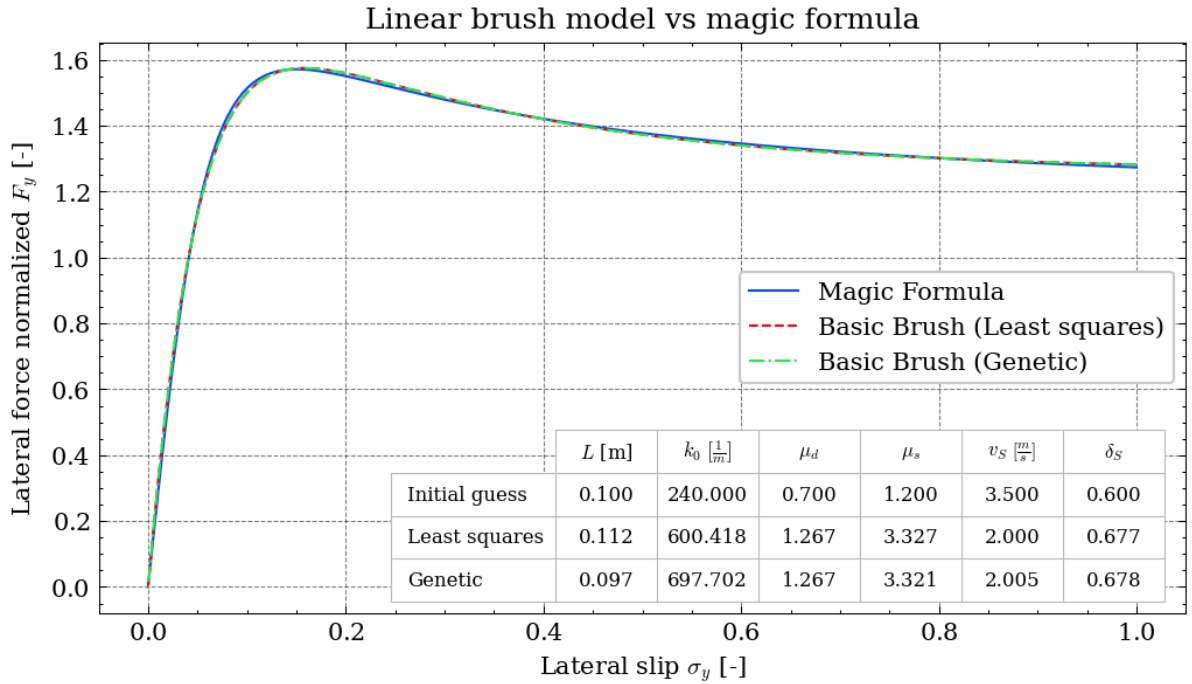


Figure 2: Comparison of the first brush model (least squares fit) against the Magic Formula reference.

As can be seen in Figure 2 the linear brush model can achieve good performance when compared to the magic formula which it was trained on. The fit using least squares was compared to the fit obtained using a genetic optimization algorithm and the local minima seems to overlap with the global minima when comparing the parameter values.

Optimization bounds sweep

We also tried sweeping over some of the bounds to see if this would yield different results. The bounds for μ_s and v_S were swept while all other parameters kept the default bounds from Table 1.

Table 2: Parameters and bound ranges used in the bounds sweep. All other parameters held at defaults from Table 1.

Symbol	Units	IC	LB swept	UB swept
μ_s	-	1.2	1.0	{2, 3, 4, 5, 6}
v_S	m/s	5	{0.1, 0.2, 0.5, 2.0}	{8, 12, 16, 20}

The results were sorted by cost (rank) then filtered to only show results with parameters that differed by at least 10%.

Table 3: Distinct parameter sets from the bounds sweep, ranked by fit cost.

Rank	Cost	L	k_0	mu_d	mu_s	v_S	delta_S
1	1.193e-3	0.112	572.674	1.237	6	0.574	0.441
2	1.193e-3	0.11	581.92	1.237	6	0.574	0.441
4	1.452e-3	0.112	577.876	1.245	5	0.852	0.491
7	1.97e-3	0.112	586.963	1.255	4	1.364	0.574
10	2.754e-3	0.112	600.106	1.267	3.327	2	0.677
17	2.754e-3	0.113	595.953	1.267	3.327	2	0.677
22	2.754e-3	0.112	604.623	1.267	3.327	2	0.677
25	3.47e-3	0.113	608.79	1.273	3	2.444	0.748
33	1.765e-2	0.116	699.986	1.303	2	5.256	1.462

As can be seen in Table 3 the optimizer either hits upper bounds on μ_s or lower bounds on v_S to get the best fit. This may be due to the magic formula not distinguishing between the relative speed between the surfaces as only the slip ratio is used and not the relative speed between the surfaces. The stribeck frictional model can provide different friction for slips at speeds so that a 50% slip at 10 m/s is different to 50% slip at 100 m/s.

Initial guess sweep

We also tried varying the initial guesses to see if the starting guess changed the minima that the optimizer landed in.

Contact length and bristle stiffness

The first initial guesses to sweep over are the contact length and bristle stiffness. They are chosen as there may exist different optima with a shorter contact patch and stiffer bristles versus a longer contact patch and softer bristles. The Swept initial conditions for the first initial condition sweep are shown in Table 4.

Table 4: Parameters varied in the initial guess sweep. All other parameters held at base IC from Table 1.

Symbol	Units	Base IC	Values swept
L	m	0.1	{0.05, 0.09, 0.12}
k_0	1/m	240	{100, 200, 300, 400, 500, 600, 700, 800}

The results were sorted by cost (rank) then filtered to only show results with parameters that differed by at least 10%.

Table 5: Fitted parameter sets from the initial condition sweep. Starting IC shows which initial guess was used; remaining columns show where the optimizer converged.

Rank	Starting IC	Cost	L	k_0	mu_d	mu_s	v_S	delta_S
1	Start 23 (L=0.12, k_0=700)	2.482e-3	0.117	574.036	1.263	3.5	1.794	0.642
2	Start 5 (L=0.05, k_0=500)	2.482e-3	0.096	697.937	1.263	3.5	1.794	0.642
3	Start 16 (L=0.09, k_0=800)	2.482e-3	0.087	768.696	1.263	3.5	1.794	0.642
6	Start 1 (L=0.05, k_0=100)	2.482e-3	0.103	648.633	1.263	3.5	1.794	0.642
13	Start 6 (L=0.05, k_0=600)	2.482e-3	0.093	722.315	1.263	3.5	1.794	0.642
15	Start 9 (L=0.09, k_0=100)	2.482e-3	0.111	601.919	1.263	3.5	1.794	0.642
24	Start 8 (L=0.05, k_0=800)	2.482e-3	0.084	800	1.263	3.5	1.794	0.642

The same cost value is reached from all initial conditions in Table 5 which indicates that reaching an optimal cost value may not dependant on the initial condition of the contact length L and the bristle stiffness k_0 .

Frictional coefficients and Stribeck parameters

The second initial guesses to sweep over are the friction parameters (μ_d , μ_s , v_S , δ_S). These are chosen to explore whether different friction starting points lead to different local minima, since the Stribeck model has multiple parameters that interact non-linearly.

Table 6: Parameters varied in the second initial guess sweep. All other parameters held at base IC from Table 1.

Symbol	Units	Base IC	Values swept
μ_d	-	0.7	{0.7, 1.5, 2.0}
μ_s	-	1.2	{1.0, 2.0, 3.5}
v_S	m/s	5	{0.1, 5.0, 12.0, 20.0}
δ_S	-	0.6	{0.1, 0.75, 1.25, 2.0}

The results were sorted by cost (rank) then filtered to only show results with fitted parameters that differed by at least 10%.

Table 7: Fitted parameter sets from the friction parameter initial condition sweep. Starting IC shows which friction initial guess was used; remaining columns show where the optimizer converged.

Rank	Starting IC	Cost	L	k_0	mu_d	mu_s	v_S	delta_S
1	Start 115 (mu_d=2.0, mu_s=2.0, v_S=0.1, delta_S=1.25)	2.482e-3	0.109	616.131	1.263	3.5	1.794	0.642
3	Start 3 (mu_d=0.7, mu_s=1.0, v_S=0.1, delta_S=1.25)	2.482e-3	0.113	595.427	1.263	3.5	1.794	0.642
5	Start 131 (mu_d=2.0, mu_s=3.5, v_S=0.1, delta_S=1.25)	2.482e-3	0.119	564.883	1.263	3.5	1.794	0.642
29	Start 96 (mu_d=1.5, mu_s=3.5, v_S=20.0, delta_S=2.0)	2.482e-3	0.115	583.152	1.263	3.5	1.794	0.642
30	Start 97 (mu_d=2.0, mu_s=1.0, v_S=0.1, delta_S=0.1)	2.482e-3	0.104	646.306	1.263	3.5	1.794	0.642
34	Start 53 (mu_d=1.5, mu_s=1.0, v_S=5.0, delta_S=0.1)	2.482e-3	0.106	634.99	1.263	3.5	1.794	0.642
65	Start 103 (mu_d=2.0, mu_s=1.0, v_S=5.0, delta_S=1.25)	2.482e-3	0.111	605.269	1.263	3.5	1.794	0.642
144	Start 100 (mu_d=2.0, mu_s=1.0, v_S=0.1, delta_S=2.0)	7.42e-1	0.12	800	1.438	1	0.382	2

The same cost value is achieved in Table 7 with the exception of start (rank 144) that converged to a bad optimum with a cost value of ≈ 200 times larger than the other optimum

Linear brush model using scaled μ_s friction

The scaled friction refers to replacing the independent μ_s parameter with $\mu_s = k_s \cdot \mu_d$, where k_s is the scale factor fitted by the optimizer. Originally these two parameters were independent, and were only related through the expression in Equation 3. The problem with independent bounds is that the optimizer could set $\mu_s \leq \mu_d$.

By parametrising $\mu_s = k_s \cdot \mu_d$ with the constraint $k_s > 1$ enforced through the parameter bounds and $\mu_s > \mu_d$ is guaranteed for every candidate solution the optimizer evaluates. In practice this also expands the effective search range for μ_s , since it is no longer capped by a fixed upper bound but scales with μ_d .

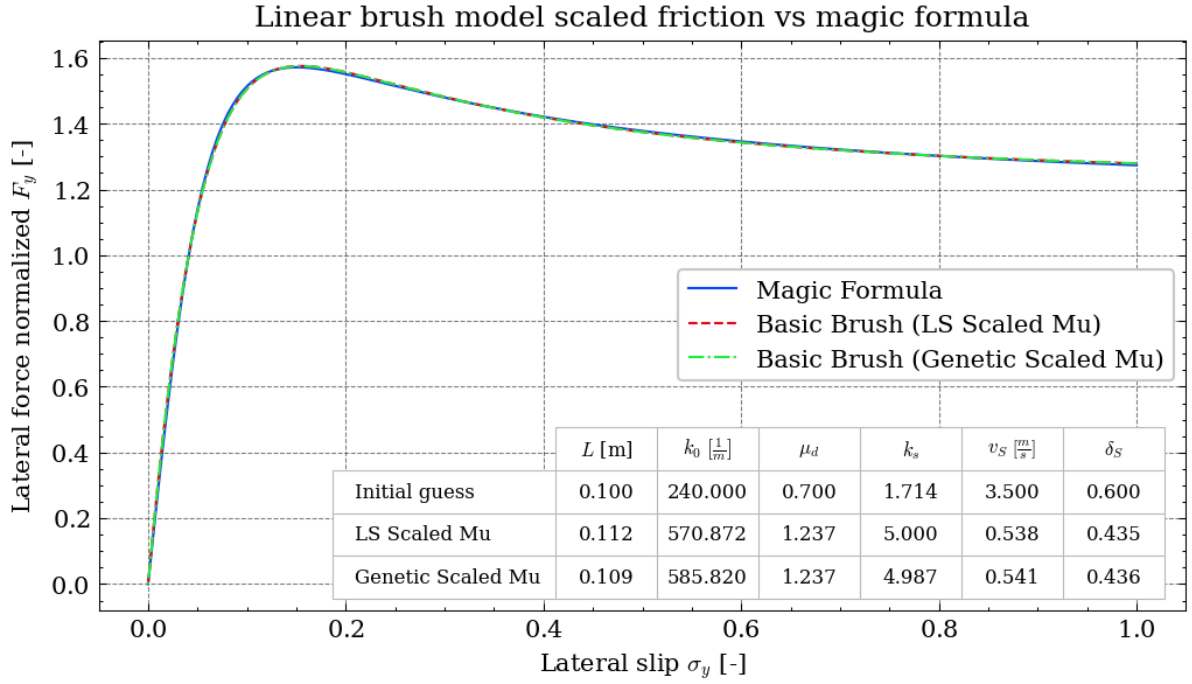


Figure 3: Comparison of the brush model (scaled μ_s , least squares fit) against the Magic Formula reference.

As seen in Figure 3 the linear brush model with scaled μ_s friction shows a good fit to the magic formula with a $\mu_d = 1.237$ and $\mu_s = 5 \cdot 1.237 = 6.185$. The fit is also alike to that of the genetic algorithm in the same plot and the least squares optimizer found the same optimum. It should be noted that both the genetic algorithm and the least squares optimizer both hit the upper bound on the friction as was the case in the previous section.

Brush model performance comparison with Luigi's code

With Pacejka's magic formula from Equation 1 as the benchmark to match, the brush model with coefficients obtained by both optimizers is shown in Figure 4.

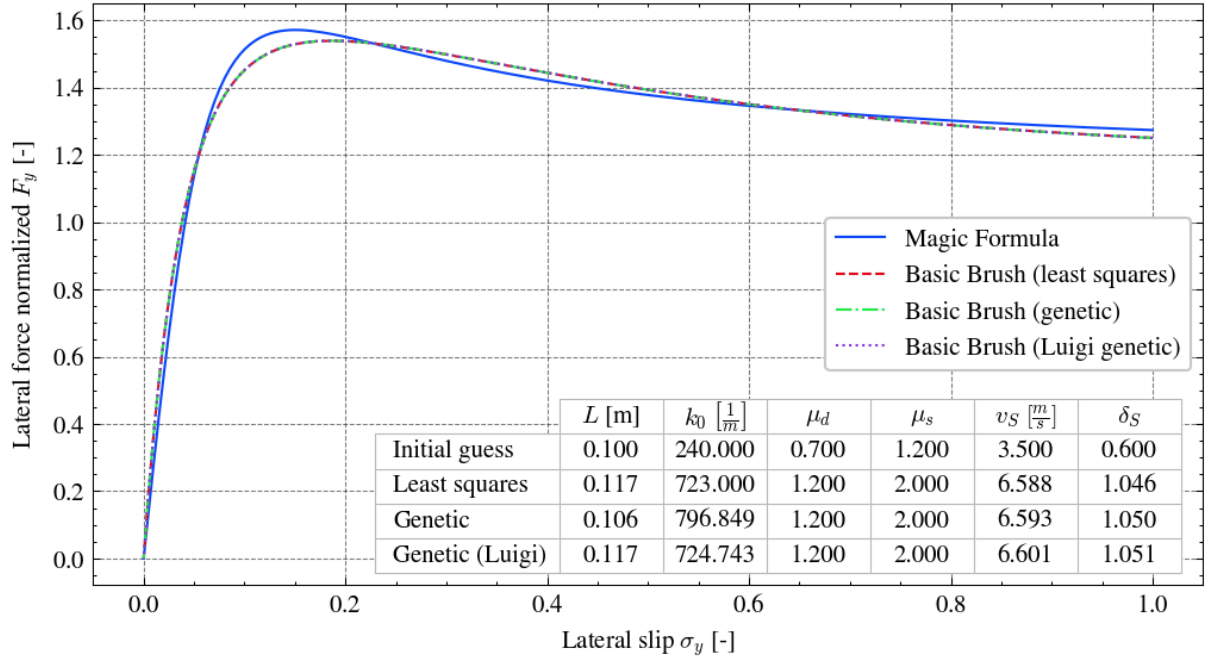


Figure 4: Lateral force curves for the Magic Formula, the least-squares-fitted brush model, and the genetic-algorithm-fitted brush model.

Our coded model compared to Luigi's model with Matlab's Genetic algorithm achieves the same fit for the given bounds and initial guess.

Least squares implementation details

Using the least squares fit, means that a local minimum is searched for the cost function:

$$F(x) = \frac{1}{2} \sum_{i=0}^{m-1} f_i(x)^2 \quad 4.$$

Where f_i is the residual. The method used for finding the local minimum is called trust region reflective algorithm, an initial guess is given with boundaries. The initial guess is evaluated, giving the current residual vector and the current loss. A jacobian is then found by perturbing the parameters in our current guess. this creates in our case a 200x6 matrix, where 200 is because we have descitized the slip ratio, and 6 because we have 6 parameters. The jacobian is used to build a local approximation, where the residual $r(x_k + p) \approx r(x_k) + J \cdot p$, where p is the proposed parameter step, and $x_k + p$ is a proposed parameter vector. Using this residual, a local approximation is build: $m_k(p) = \frac{1}{2} \|r(x_k) + J \cdot p\|^2$. The approximation is only reliable near our currently parameter vector x_k , since the jacobian is found by perturbation. Thus we define a trusted region.

$$\|p\| \leq \Delta$$

This also complies with the bounds that is given, meaning $l \leq x_k + p \leq u$. SciPy then tries to find the best step p , such that the approximate loss is reduced

$$\min_p \frac{1}{2} \|r(x_k) + J \cdot p\|^2 \quad 5.$$

for which it will then propose $x_{\text{trail}} = x_k + p$ with this p and x_{trail} , the predicted improvement is evaluated

$$\rho = \frac{m_k(0) - m_k(p)}{F(x_k) - F(x_{\text{trail}})} \quad 6.$$

If this ratio is good, it means that the local approximation predicts the real behavior well and will then take $x_{k+1} = x_{\text{trail}}$, if the ratio is bad then $x_{k+1} = x_k$. now SciPy updates the trust region Δ , where if the prediction was good, Δ will grow, thus letting the model take larger steps. If the prediction was bad, Δ decreases and the model will be more cautious. If a parameter is close to a bound, and the proposed change would violate this bound, the model will either reject the step, and reduce the trust region. Reduce the step for that specific parameter. Or try to change the step in the other parameters to compensate for the lack of step of the limited parameter. This is the full algorithm that will be repeated until it converges and a criteria is met.

Bibliography

- [1] M. Guiggiani, *The science of vehicle dynamics : handling, braking, and ride of road and race cars*, Third edition. Cham: Springer, 2022.